

[E3] Atividade Prática
Dedução de Hazards e Redução de Bolhas

Aluno(a): _____

RA: _____

1. Definições

O Projeto PRISMA é uma iniciativa para tornar mais visual e interativo o processo de estudo e desenvolvimento de componentes para um processador, fornecendo uma base pronta para que testes e experimentos possam ser realizados na arquitetura RISC-V. A visualização dos dados que transitam em conexões e a análise de componentes e portas lógicas que constituem o processador é possível por meio do uso do simulador visual de circuitos digitais Logisim-Evolution.

A implementação disponível no projeto a ser utilizada nessa atividade é:

- **Pipeline simples ([GitHub](#))**

Nessa arquitetura, o caminho de dados é separado em cinco estágios usando bancos de registradores intermediários, que servem para armazenar informações produzidas em um estágio anterior que serão necessárias na etapa seguinte. Nessa modificação, o período de clock é dividido entre o número de etapas e, assim, cinco instruções podem ser carregadas simultaneamente no caminho de dados. Dessa forma, há maior *throughput* de instruções do que no ciclo único. Entretanto, alguns conflitos de dependência de dados que surgem nessa implementação precisam ser resolvidos no código com a adição de instruções NOP pelo software, que servem para dar um tempo entre uma instrução e outra, e em casos extremos, faz com que o potencial ganho de desempenho seja anulado.

As instruções suportadas pelo Projeto PRISMA estão descritas na Tabela 1, e todas seguem as especificações da arquitetura RISC-V.

Aritmética/Lógicas (AL)	AL com imediato	Memória	Desvios
add	addi	lb	beq
sub	xori	lh	bne
xor	ori	lw	blt
or	andi	lbu	bge
and	slli	lhu	bltu
sll	srli	sb	bgeu
srl	srai	sh	jal
sra	slti	sw	jalr
	sltiu		
	lui		
	auipc		

Tabela 1 - Instruções suportadas

Conceitos que serão utilizados na atividade:

- **Sombra de controle (*branch delay slot*)**: quantidade de instruções que o processador busca e começa a executar "por engano" logo após uma instrução de desvio enquanto ainda está tentando descobrir para onde deve saltar.

2. Atividade

Escreva um código com manipulação de valores na memória e no banco de registradores para execução no caminho de dados *Pipeline* simples do Projeto PRISMA. Utilize o [CREATOR](#) para auxílio na escrita do código ou o simulador [Venus](#)

para conversão das instruções *assembly* para linguagem de máquina. O algoritmo deve englobar ao menos 2 ocorrências diferentes de cada cenário:

1. Desvio incondicional (`jal`, `jalr`).
2. Desvio condicional verdadeira (`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`).
3. Desvio condicional falsa (`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`).

Considerando a arquitetura de *Pipeline* simples, para cada quebra de fluxo deve-se inserir uma quantidade adequada de `NOPs`, ou seja, uma sequência de operações vazias para proteger o código da sombra de controle gerada pelo hardware e evitar a execução indevida de instruções. **Tente reduzir os `NOPs` ao mínimo necessário.** A coerência entre o tamanho real da sombra e a quantidade de instruções vazias que você aplicou ao algoritmo que escolheu será avaliada na nota final.

Crie uma tabela como a seguinte (Tabela 2) e, para cada instrução de desvio do seu algoritmo que precise de bolha, adicione uma linha descrevendo o tipo da operação e a quantidade mínima de bolhas para a execução correta.

Instrução	Tipo da operação	Bolhas
6. <code>jalr zero, 0(ra)</code>	Desvio incondicional	2
9. <code>beq t0, t1, destino</code>	Desvio condicional falsa	1

Tabela 2 - Bolhas necessárias em desvios

3. Relatório

Prepare um relatório que analise os resultados listados na tabela que criou com base na Tabela 2 e justifique a diferença entre o tamanho de sombra em cada

tipo de operação, descrevendo o impacto da divisão do caminho de dados em bancos de registradores de estágios do *Pipeline* simples em cada uma delas.